МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 1 ЗА ТЕМОЮ:
Реалізація rule-based системи

Група ІАВ - 21
Курс 2
Студентка Кодовбецька В. І.
Дата оформлення 10.10.2026

Київ - 2026

Лабораторна робота №1. Реалізація rule-based системи

Завдання

1. Реалізувати rule-based систему в Python.
2. Завантажити базу фактів і правил.
3. Реалізувати механізм логічного виведення.

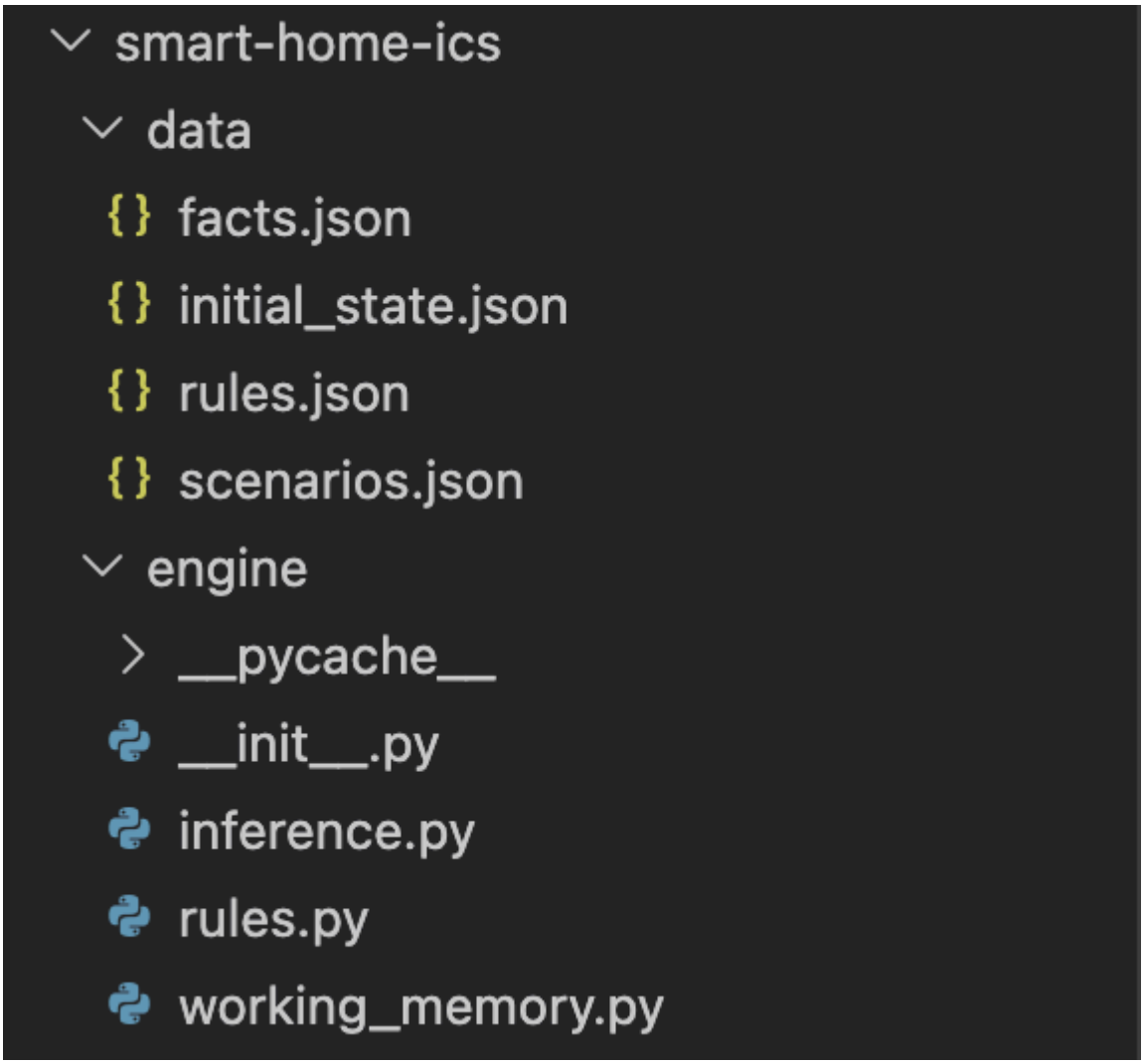
Очікуваний результат: працюючий прототип rule-based системи, який може робити висновки на основі фактів і правил.

Хід виконання завдання:

У цій роботі реалізовано ядро експертної системи керування мікрокліматом розумного будинку, спроєктованої в практичних роботах №1–3. Програму написано на Python без сторонніх бібліотек, тому вона запускається на будь-якому комп'ютері з Python 3.10 і новіше. Факти й правила не вшиті в код, а лежать окремими JSON-файлами, щоб у наступних лабораторних їх можна було доповнювати, не переписуючи програму.

1. Реалізація rule-based системи в Python

Проєкт розбито на окремі компоненти, як у блок-схемі з практичної роботи №1: база знань (папка data), робоча пам'ять і механізм виведення (папка engine) та точка запуску main.py.



rules.json - 10 правил з ПР3
facts.json - база фактів з ПР2
initial_state.json - початковий стан пристроїв
scenarios.json - тестові ситуації
working_memory.py - робоча пам'ять
rules.py - правила та перевірка умов
inference.py - механізм виведення
main.py - запуск із консолі

1) Робоча пам'ять (working_memory.py) – зберігає поточні факти у вигляді «об'єкт.атрибут → значення», наприклад «Кімната.температура → 28».
2) Правила (rules.py) – клас Rule зберігає назву, пріоритет, вагу, умову й дії правила, вміє перевірити умову, виконати дії та пояснити, яких фактів бракує, щоб правило спрацювало.
3) Механізм виведення (inference.py) – функція run, яка циклічно застосовує правила до робочої пам'яті.
4) Точка запуску (main.py) – завантажує файли, прогоняє сценарії й виводить хід міркувань і стан пристроїв.

2. Завантаження бази фактів і правил

Базу фактів записано у форматі «об'єкт – атрибут – значення», як у практичній роботі №2. Кожен факт – окремий рядок JSON:

```
{"object": "Кімната", "attribute": "температура", "value": 28},
{"object": "Кімната", "attribute": "рівень_CO2", "value": 600},
{"object": "Мешканці", "attribute": "є_люди", "value": "ні"},
```

Перед фактами завантажується початковий стан (initial_state.json): режим «комфортний», криза «ні», усі пристрої вимкнені. Тестові ситуації в scenarios.json беруть за основу facts.json і змінюють у ньому лише потрібні факти.

Правила перенесено з формального запису в додатку Б практичної роботи №3. Умова – це або окреме порівняння, або група умов «all» (І) чи «any» (АБО). Для прикладу правило R5:

```
{
  "id": "R5", "name": "Високий CO2", "salience": 80, "weight": 0.8,
  "if": {"all": [
    {"fact": "Кімната.рівень_CO2", "op": ">", "value": 1000},
    {"fact": "Мешканці.є_люди", "op": "==", "value": "так"},
    {"fact": "Режим.криза", "op": "==", "value": "ні"},
    {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
  ]},
```

```
  "then": [
    {"set": "Вікна.стан", "value": "відкрито"}
  ]
},
```

Дія може мати власну умову. Так зроблено в R1: вікна закриваються завжди, а кондиціонер переходить у рециркуляцію лише тоді, коли живлення від мережі. Якщо в робочій пам'яті немає факту, на який посилається умова, умова вважається невиконаною, і програма не падає.

3. Механізм логічного виведення

Реалізовано пряме виведення (forward chaining): система йде від фактів до висновків. Кожен цикл складається з трьох кроків, показаних на рисунку 1.

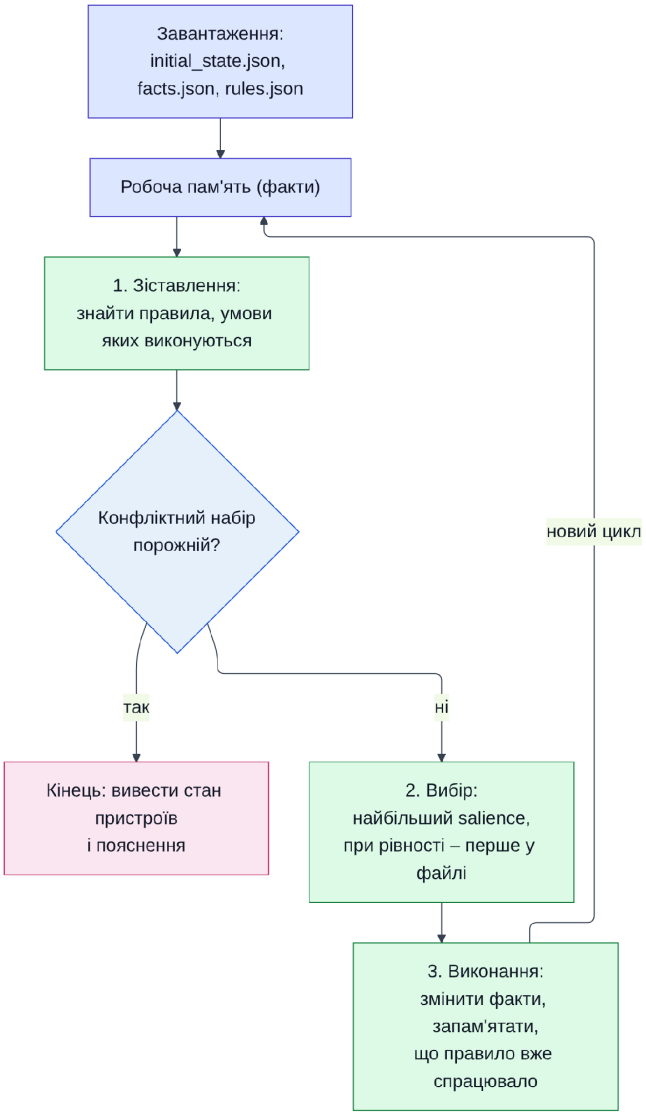


Рисунок 1 – Цикл роботи механізму виведення

1) Зіставлення. Перевіряються умови всіх правил, і ті, що виконуються, потрапляють у конфліктний набір.
2) Вибір. З набору береться правило з найбільшим salience. Якщо пріоритети однакові, спрацьовує те, що стоїть раніше у файлі.

3) Виконання. Права частина правила змінює факти в робочій пам'яті, і починається новий цикл, бо нові факти можуть увімкнути або вимкнути інші правила.

Кожне правило спрацьовує лише один раз (рефракція), тому система не може зациклитись. Виведення зупиняється, коли конфліктний набір порожній. Для кожного циклу програма друкує конфліктний набір, обране правило, факти, через які воно спрацювало, і зміни, тобто пояснює своє рішення. З ключем --explain вона ще й показує, яких умов не вистачило правилам, що не спрацювали.

Результати роботи

Систему перевірено на чотирьох сценаріях з файлу scenarios.json.

Сценарій	Спрацювали правила	Результат
1. Приклад з ПР2: літо, 28 °C, вдома лише тварини	R6, R10	помірне охолодження, поїлка увімкнена, режим економний
2. Блекаут посеред спекотного дня	R2	усі пристрої вимкнено, режим сну
3. Обстріл, люди вдома, CO_2 1300 ppm	R1	криза, вікна в ізоляції, кондиціонер на рециркуляції; R5 і R7 заблоковано
4. Мережа повернулась, зима, 16 °C, сніг	R3, R8, R9	режим комфортний, обігрівач і тепла підлога увімкнені

Перший сценарій – це база фактів із практичної роботи №2. Як і очікувалось, спрацювали R6 і R10, причому R6 першим через вищий пріоритет (рисунок 2).

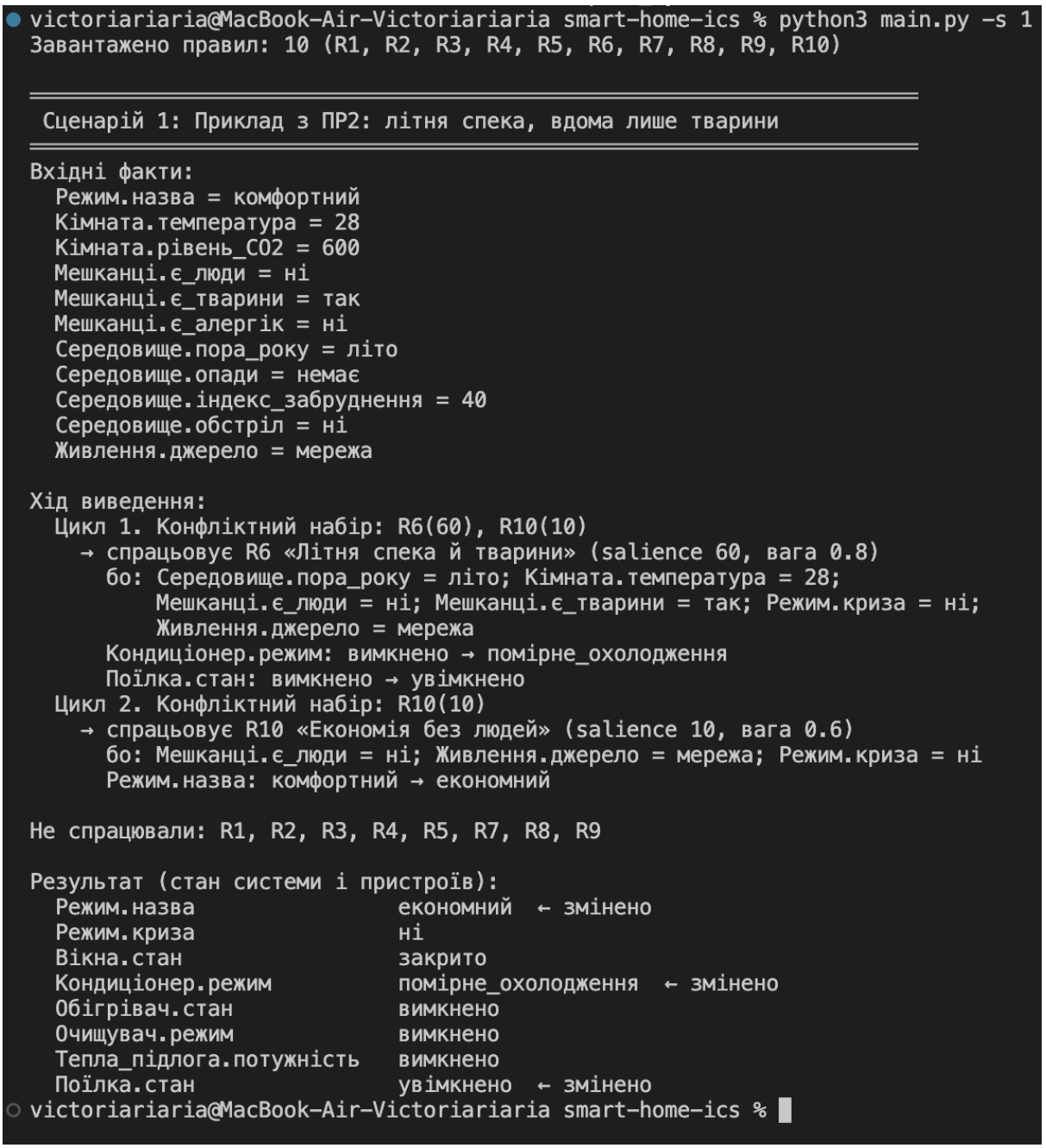


Рисунок 2 – Сценарій 1: приклад з практичної роботи №2

У другому сценарії до блекауту працювали кондиціонер, поїлка й очищувач. R2 вимкнув їх і перевів систему в режим сну, а решта правил не спрацювала, бо всі вони вимагають живлення від мережі (рисунок 3).

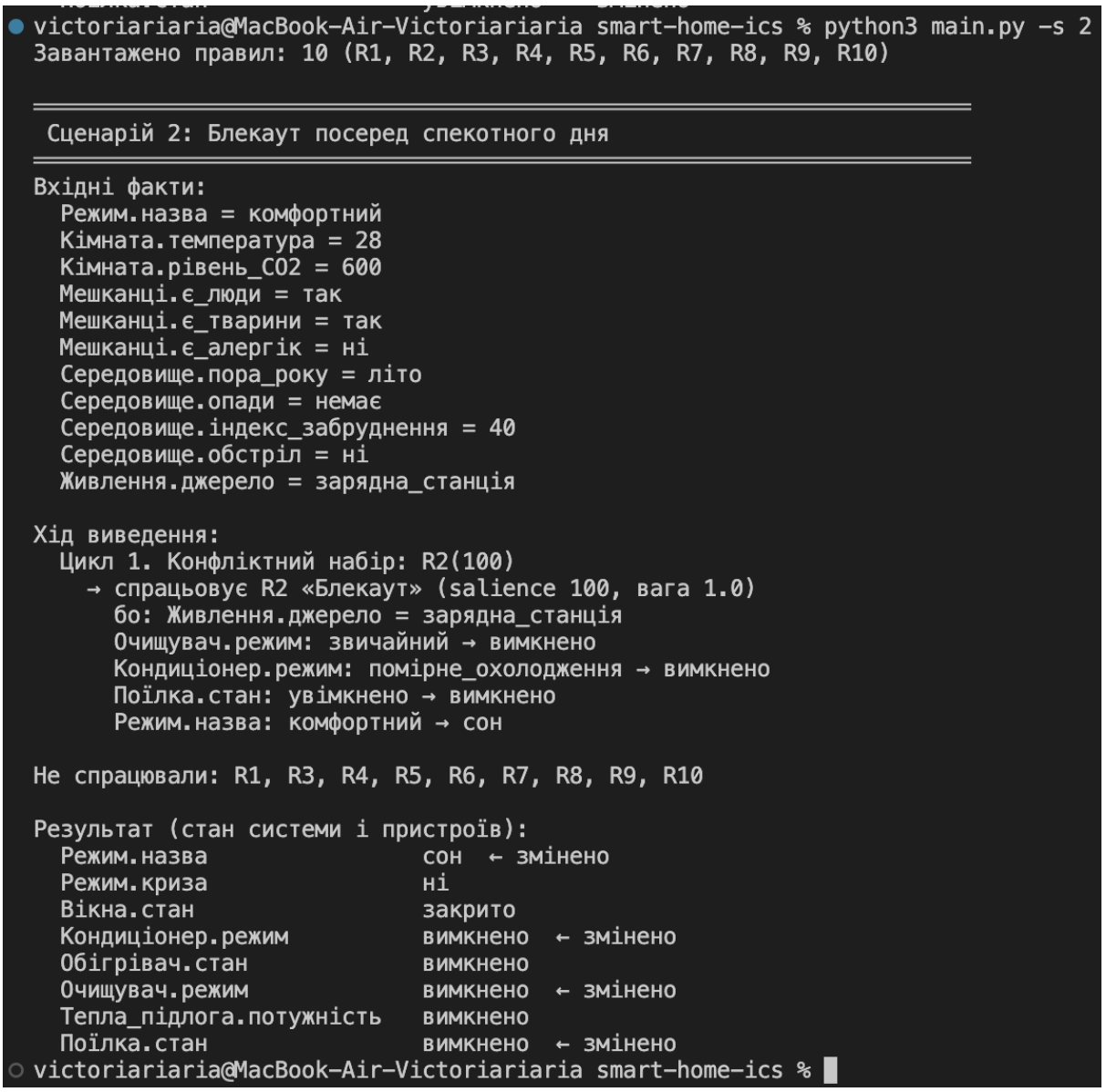


Рисунок 3 – Сценарій 2: блекаут

Третій сценарій найцікавіший. На початку в конфліктному наборі опинились одразу три правила: R1, R5 (відкрити вікна через CO_2) і R7 (охолодження). Першим спрацювало R1 як найважливіше й записало факт «криза = так». У наступному циклі R5 і R7 вже не пройшли перевірку, бо обидва вимагають «немає кризи». Так конфлікт між безпекою й комфортом, описаний у практичній роботі №3, розв'язався автоматично. Режим --explain показує причину для кожного правила (рисунок 4).

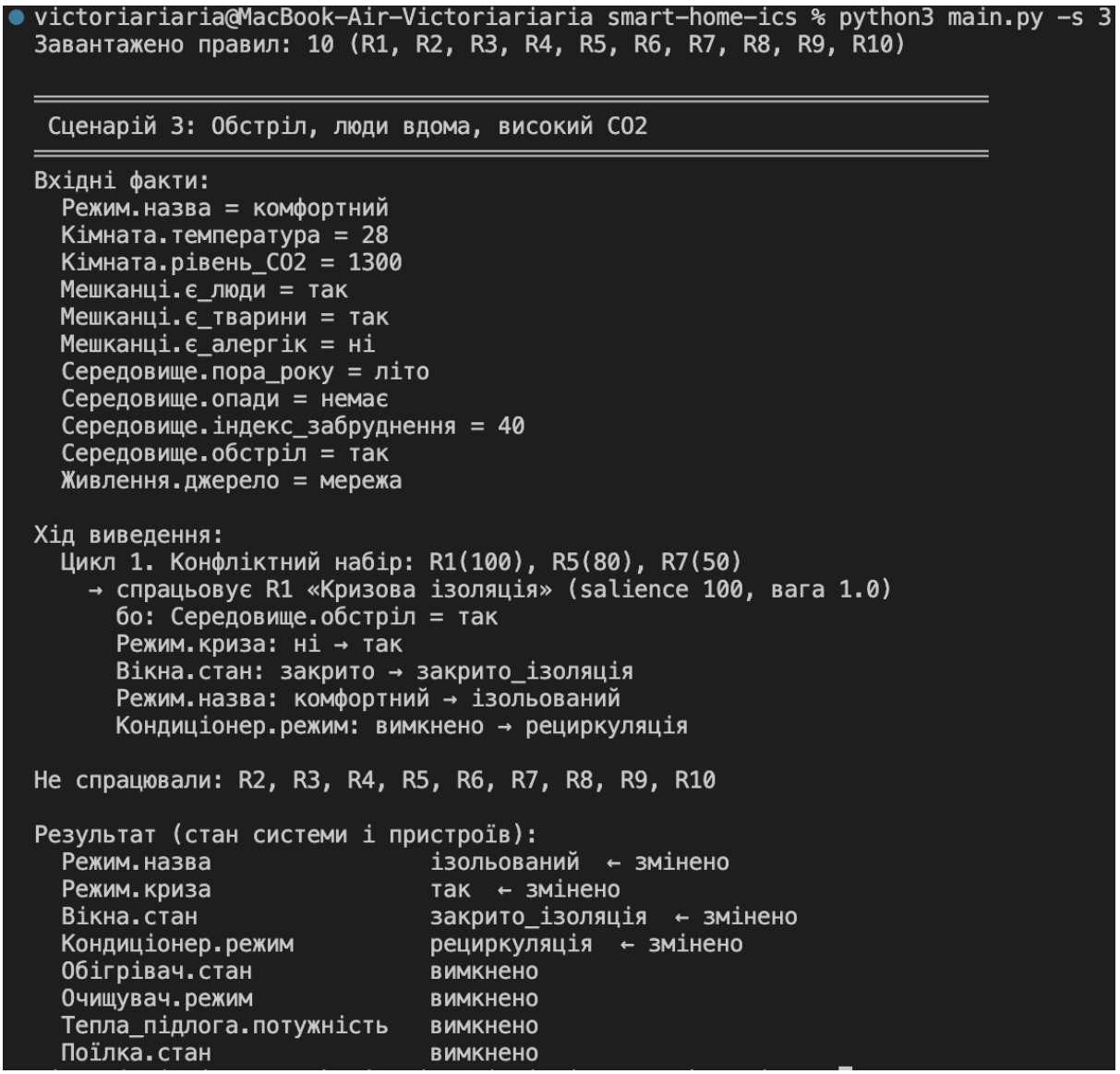


Рисунок 4 – Сценарій 3: обстріл, запуск з поясненням

Четвертий сценарій показує ланцюжок виведення на кількох циклах: спочатку R3 повернуло комфортний режим після сну, потім за пріоритетом спрацювали R8 (холодно, люди вдома) і R9 (сніг) (рисунок 5).

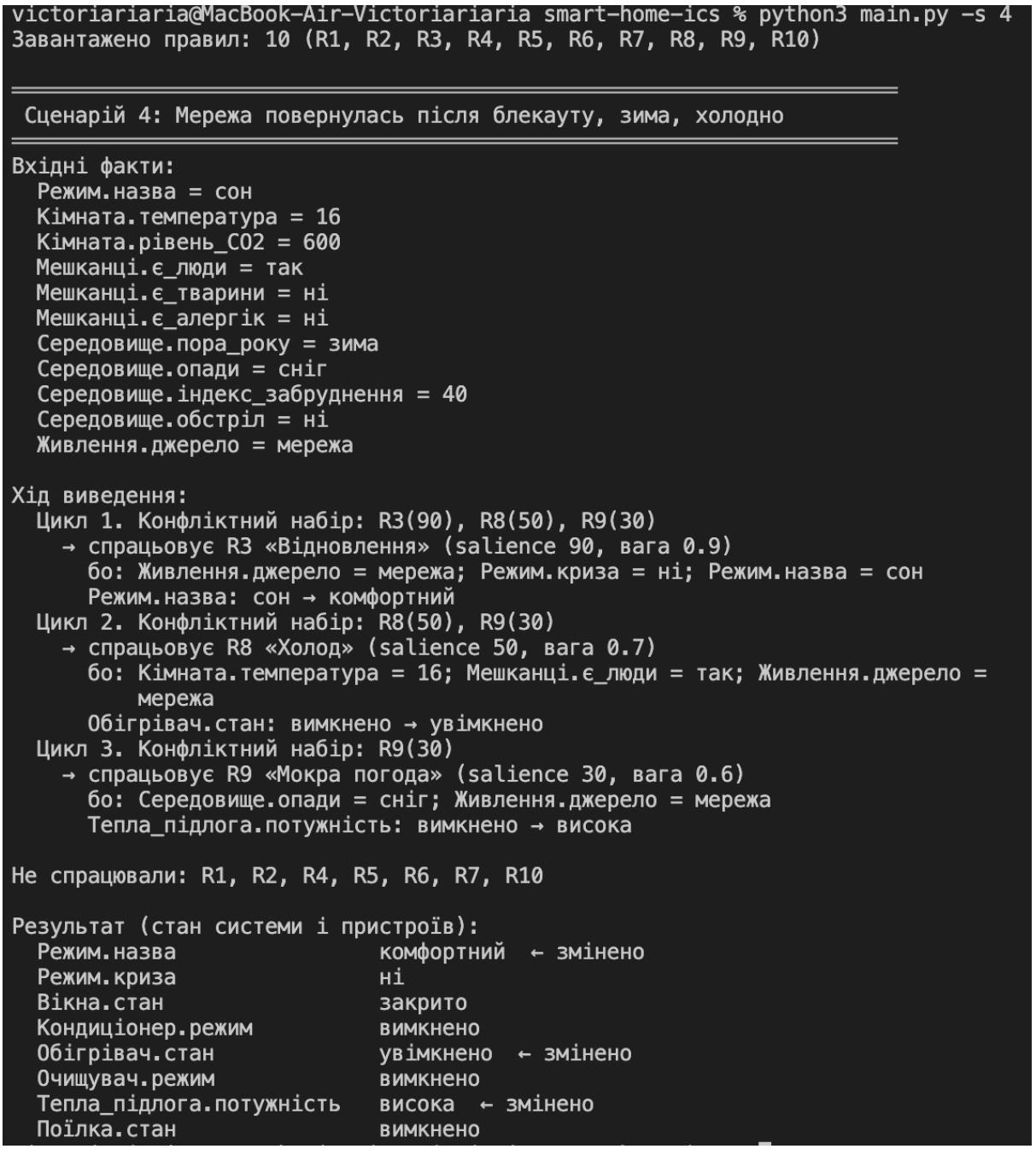


Рисунок 5 – Сценарій 4: відновлення мережі взимку

Висновки

У ході роботи реалізовано rule-based систему на Python, яка завантажує базу фактів і базу з десяти правил із JSON-файлів і робить висновки прямим виведенням. Конфлікти між правилами розв'язуються пріоритетами (salience), а факт «криза», який виводить R1, блокує небезпечні дії інших правил. Система пояснює кожне рішення: яке правило

спрацювало, через які факти і що змінилось. Перевірка на чотирьох сценаріях дала очікувані результати, зокрема для бази фактів із практичної роботи №2. Ваги правил поки лише зберігаються і не впливають на висновок, вони знадобляться в лабораторній роботі №2 для обчислення ступеня впевненості.

Додатки

Додаток А – Програмний код

engine/working_memory.py

```
"""Робоча пам'ять: поточні факти у вигляді «об'єкт.атрибут -> значення»."""


class WorkingMemory:
    def __init__(self):
        self.facts = {}

    @staticmethod
    def key(obj, attr):
        return f"{obj}.{attr}"

    def load(self, triples):
        """Завантажує список фактів формату {"object", "attribute", "value"}."""
        for t in triples:
            self.facts[self.key(t["object"], t["attribute"])] = t["value"]

    def get(self, key, default=None):
        return self.facts.get(key, default)

    def has(self, key):
        return key in self.facts

    def set(self, key, value):
        """Записує факт. Повертає (старе значення, змінилось чи ні)."""
        old = self.facts.get(key)
        self.facts[key] = value
        return old, old != value

    def snapshot(self):
        return dict(self.facts)
```

engine/rules.py

```
"""Продукційні правила «ЯКЩО - ТО»: завантаження з JSON і перевірка умов."""
import json

OPERATORS = {
    "==": lambda a, b: a == b,
    "!=": lambda a, b: a != b,
```

```
    ">": lambda a, b: a > b,
    "<": lambda a, b: a < b,
    ">=": lambda a, b: a >= b,
    "<=": lambda a, b: a <= b,
    "in": lambda a, b: a in b,
}


def check(cond, wm):
    """Перевіряє умову. Умова - це атом {"fact", "op", "value"}
    або група {"all": [...]} (І) чи {"any": [...]} (АБО)."""
    if "all" in cond:
        return all(check(c, wm) for c in cond["all"])
    if "any" in cond:
        return any(check(c, wm) for c in cond["any"])
    if not wm.has(cond["fact"]):
        return False          # невідомий факт - умова не виконується
    try:
        return OPERATORS[cond["op"]](wm.get(cond["fact"]), cond["value"])
    except TypeError:         # наприклад, порівняння числа з рядком
        return False


def atoms(cond):
    """Усі атомарні умови всередині складної умови."""
    if "all" in cond:
        return [a for c in cond["all"] for a in atoms(c)]
    if "any" in cond:
        return [a for c in cond["any"] for a in atoms(c)]
    return [cond]


def describe(cond):
    value = cond["value"]
    if isinstance(value, list):
        value = " / ".join(map(str, value))
    op = "∈" if cond["op"] == "in" else cond["op"].replace("==", "=")
    return f"{cond['fact']} {op} {value}"


class Rule:
    def __init__(self, data, order):
        self.id = data["id"]
        self.name = data["name"]
        self.salience = data.get("salience", 0)
```

```
        self.weight = data.get("weight", 1.0)
        self.condition = data["if"]
        self.actions = data["then"]
        self.order = order    # порядок у файлі, якщо пріоритети однакові

    def matches(self, wm):
        return check(self.condition, wm)

    def reasons(self, wm):
        """Факти, через які правило спрацювало (виконані атомарні умови)."""
        return [f"{a['fact']} = {wm.get(a['fact'])}"
                for a in atoms(self.condition) if check(a, wm)]

    def failed(self, wm):
        """Умови, які не виконались (для пояснення, чому правило мовчить)."""
        out = []
        for a in atoms(self.condition):
            if not check(a, wm):
                out.append(f"потрібно {describe(a)}, а є {wm.get(a['fact'],
'невідомо')}")
        return out

    def fire(self, wm):
        """Виконує праву частину правила. Повертає список змін."""
        changes = []
        for act in self.actions:
            if "if" in act and not check(act["if"], wm):
                continue
            old, changed = wm.set(act["set"], act["value"])
            if changed:
                changes.append((act["set"], old, act["value"]))
        return changes

    def __repr__(self):
        return f"{self.id}({self.salience})"


def load_rules(path):
    with open(path, encoding="utf-8") as f:
        return [Rule(d, i) for i, d in enumerate(json.load(f))]
```

engine/inference.py

```
"""Механізм логічного виведення: прямий ланцюжок (forward chaining).
```

```
Кожен цикл має три кроки:
 1) зіставлення – знайти всі правила, умови яких виконуються (конфліктний набір);
 2) вибір – узяти правило з найбільшим salience (при рівності – перше у файлі);
 3) виконання – змінити факти в робочій пам'яті.
Кожне правило спрацьовує не більше одного разу (рефракція), тому система
не зациклюється. Виведення зупиняється, коли конфліктний набір порожній.
"""
from dataclasses import dataclass, field


@dataclass
class Step:
    cycle: int
    agenda: list          # конфліктний набір цього циклу
    rule: object          # правило, яке спрацювало
    reasons: list         # факти, через які воно спрацювало
    changes: list = field(default_factory=list)


def run(rules, wm, max_cycles=50):
    fired = set()
    trace = []
    for cycle in range(1, max_cycles + 1):
        agenda = [r for r in rules if r.id not in fired and r.matches(wm)]
        if not agenda:
            break
        agenda.sort(key=lambda r: (-r.salience, r.order))
        rule = agenda[0]
        reasons = rule.reasons(wm)
        changes = rule.fire(wm)
        fired.add(rule.id)
        trace.append(Step(cycle, agenda, rule, reasons, changes))
    return trace
```

main.py

```
"""Rule-based система керування мікрокліматом розумного будинку (ЛР1).

Запуск:
 python main.py                 – усі сценарії з data/scenarios.json
 python main.py -s 3            – лише сценарій 3
 python main.py -s 3 --explain  – плюс пояснення, чому інші правила не спрацювали
 python main.py -f my.json      – власний файл фактів
```

```
"""
import argparse
import json
import sys
import textwrap
from pathlib import Path

from engine import WorkingMemory, load_rules, run

DATA = Path(__file__).parent / "data"
DEVICES = ["Режим.назва", "Режим.криза", "Вікна.стан", "Кондиціонер.режим",
           "Обігрівач.стан", "Очищувач.режим", "Тепла_підлога.потужність",
"Поїлка.стан"]


def read_json(name):
    path = Path(name) if Path(name).exists() else DATA / name
    with open(path, encoding="utf-8") as f:
        return json.load(f)


def build_memory(scenario):
    wm = WorkingMemory()
    wm.load(read_json("initial_state.json"))
    wm.load(read_json(scenario.get("facts_file") or scenario.get("base",
"facts.json")))
    wm.load(scenario.get("facts", []))
    return wm


def show(name, rules, wm, explain=False):
    line = "=" * 70
    print(f"\n{line}\n {name}\n{line}")
    print("Вхідні факти:")
    for k, v in wm.snapshot().items():
        if k not in DEVICES or k == "Режим.назва":
            print(f"  {k} = {v}")

    before = wm.snapshot()
    trace = run(rules, wm)

    print("\nХід виведення:")
    if not trace:
        print("  жодне правило не спрацювало")
    for st in trace:
```

```
        agenda = ", ".join(repr(r) for r in st.agenda)
        print(f"  Цикл {st.cycle}. Конфліктний набір: {agenda}")
        print(f"    → спрацьовує {st.rule.id} «{st.rule.name}» "
              f"(salience {st.rule.salience}, вага {st.rule.weight})")
        print(textwrap.fill("; ".join(st.reasons), width=78,
                            initial_indent="      бо: ", subsequent_indent="
"))
        for key, old, new in st.changes:
            print(f"      {key}: {old} → {new}")
        if not st.changes:
            print("      (факти вже мали ці значення)")

    fired = {st.rule.id for st in trace}
    silent = [r for r in rules if r.id not in fired]
    print("\nНе спрацювали: " + ", ".join(r.id for r in silent))
    if explain:
        for r in silent:
            print(textwrap.fill("; ".join(r.failed(wm)), width=78,
                                initial_indent=f"  {r.id} «{r.name}»: ",
subsequent_indent="      "))

    print("\nРезультат (стан системи і пристроїв):")
    for k in DEVICES:
        mark = "  ← змінено" if before.get(k) != wm.get(k) else ""
        print(f"  {k:26} {wm.get(k)}{mark}")
    return trace


def main():
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")

    p = argparse.ArgumentParser(description="Rule-based система розумного будинку")
    p.add_argument("-s", "--scenario", type=int, help="номер сценарію (з 1)")
    p.add_argument("-f", "--facts", help="власний JSON-файл фактів")
    p.add_argument("--explain", action="store_true", help="пояснити, чому правила не
спрацювали")
    args = p.parse_args()

    rules = load_rules(DATA / "rules.json")
    print(f"Завантажено правил: {len(rules)} ({', '.join(r.id for r in rules)})")

    if args.facts:
        scenarios = [{"name": f"Факти з файлу {args.facts}", "facts_file":
args.facts}]
```

```
    else:
        scenarios = read_json("scenarios.json")
        if args.scenario:
            scenarios = [scenarios[args.scenario - 1]]

    for i, sc in enumerate(scenarios, 1):
        num = args.scenario or i
        title = sc["name"] if args.facts else f"Сценарій {num}: {sc['name']}"
        show(title, rules, build_memory(sc), args.explain)


if __name__ == "__main__":
    main()
```

Додаток Б – База правил (data/rules.json)

```
[
  {
    "id": "R1", "name": "Кризова ізоляція", "salience": 100, "weight": 1.0,
    "if": {"any": [
      {"fact": "Середовище.індекс_забруднення", "op": ">", "value": 100},
      {"fact": "Середовище.обстріл", "op": "==", "value": "так"}
    ]},
    "then": [
      {"set": "Режим.криза", "value": "так"},
      {"set": "Вікна.стан", "value": "закрито_ізоляція"},
      {"if": {"fact": "Живлення.джерело", "op": "==", "value": "мережа"},
       "set": "Режим.назва", "value": "ізольований"},
      {"if": {"fact": "Живлення.джерело", "op": "==", "value": "мережа"},
       "set": "Кондиціонер.режим", "value": "рециркуляція"}
    ]
  },
  {
    "id": "R2", "name": "Блекаут", "salience": 100, "weight": 1.0,
    "if": {"fact": "Живлення.джерело", "op": "==", "value": "зарядна_станція"},
    "then": [
      {"set": "Очищувач.режим", "value": "вимкнено"},
      {"set": "Кондиціонер.режим", "value": "вимкнено"},
      {"set": "Обігрівач.стан", "value": "вимкнено"},
      {"set": "Тепла_підлога.потужність", "value": "вимкнено"},
      {"set": "Поїлка.стан", "value": "вимкнено"},
      {"set": "Режим.назва", "value": "сон"}
    ]
  },
  {
    "id": "R3", "name": "Відновлення", "salience": 90, "weight": 0.9,
    "if": {"all": [
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"},
      {"fact": "Режим.криза", "op": "==", "value": "ні"},
      {"fact": "Режим.назва", "op": "in", "value": ["сон", "ізольований"]}
    ]},
    "then": [
      {"set": "Режим.назва", "value": "комфортний"}
    ]
  },
  {
    "id": "R4", "name": "Весняна алергія", "salience": 80, "weight": 0.9,
    "if": {"all": [
      {"fact": "Середовище.пора_року", "op": "==", "value": "весна"},
      {"fact": "Мешканці.є_алергік", "op": "==", "value": "так"},
      {"fact": "Мешканці.є_тварини", "op": "==", "value": "так"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Очищувач.режим", "value": "максимальний"}
    ]
  },
  {
    "id": "R5", "name": "Високий CO2", "salience": 80, "weight": 0.8,
    "if": {"all": [
      {"fact": "Кімната.рівень_CO2", "op": ">", "value": 1000},
      {"fact": "Мешканці.є_люди", "op": "==", "value": "так"},
      {"fact": "Режим.криза", "op": "==", "value": "ні"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Вікна.стан", "value": "відкрито"}
    ]
  },
  {
    "id": "R6", "name": "Літня спека й тварини", "salience": 60, "weight": 0.8,
    "if": {"all": [
      {"fact": "Середовище.пора_року", "op": "==", "value": "літо"},
      {"fact": "Кімната.температура", "op": ">", "value": 26},
      {"fact": "Мешканці.є_люди", "op": "==", "value": "ні"},
      {"fact": "Мешканці.є_тварини", "op": "==", "value": "так"},
      {"fact": "Режим.криза", "op": "==", "value": "ні"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Кондиціонер.режим", "value": "помірне_охолодження"},
      {"set": "Поїлка.стан", "value": "увімкнено"}
    ]
  },
```

```
  {
    "id": "R7", "name": "Спека з людьми", "salience": 50, "weight": 0.7,
    "if": {"all": [
      {"fact": "Кімната.температура", "op": ">", "value": 26},
      {"fact": "Мешканці.є_люди", "op": "==", "value": "так"},
      {"fact": "Режим.криза", "op": "==", "value": "ні"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Кондиціонер.режим", "value": "помірне_охолодження"}
    ]
  },
  {
    "id": "R8", "name": "Холод", "salience": 50, "weight": 0.7,
    "if": {"all": [
      {"fact": "Кімната.температура", "op": "<", "value": 18},
      {"fact": "Мешканці.є_люди", "op": "==", "value": "так"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Обігрівач.стан", "value": "увімкнено"}
    ]
  },
  {
    "id": "R9", "name": "Мокра погода", "salience": 30, "weight": 0.6,
    "if": {"all": [
      {"fact": "Середовище.опади", "op": "in", "value": ["дощ", "сніг"]},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"}
    ]},
    "then": [
      {"set": "Тепла_підлога.потужність", "value": "висока"}
    ]
  },
  {
    "id": "R10", "name": "Економія без людей", "salience": 10, "weight": 0.6,
    "if": {"all": [
      {"fact": "Мешканці.є_люди", "op": "==", "value": "ні"},
      {"fact": "Живлення.джерело", "op": "==", "value": "мережа"},
      {"fact": "Режим.криза", "op": "==", "value": "ні"}
    ]},
    "then": [
      {"set": "Режим.назва", "value": "економний"}
    ]
  }
]
```